PYSPARK COMPLETE NOTES

PHASE 6

Writing Data & Delta Lake

Save DataFrames as files and tables, then make them reliable with Delta Lake and Unity Catalog.

YOU WILL

- ✓ Write a DataFrame with df.write.format().mode().save()
- ✓ Name the four write modes and say exactly what each does
- ✓ Explain why Parquet beats CSV for analytics
- ✓ Tell managed tables from external tables
- ✓ Explain Delta Lake, ACID and the transaction log
- ✓ Choose save vs saveAsTable and append vs overwrite vs merge
- ✓ Use schema enforcement, evolution, history, time travel and OPTIMIZE
- ✓ Describe Unity Catalog and managed vs external tables

START HERE IF

Phases 1-5 (you need a DataFrame worth saving).

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 6 · Writing Data & Delta Lake

Save DataFrames as files and tables, then make them reliable with Delta Lake and Unity Catalog.

YOU WILL BE ABLE TO

- ✓ Write a DataFrame with df.write.format().mode().save()
- ✓ Name the four write modes and say exactly what each does
- ✓ Explain why Parquet beats CSV for analytics
- ✓ Tell managed tables from external tables
- ✓ Explain Delta Lake, ACID and the transaction log
- ✓ Choose save vs saveAsTable and append vs overwrite vs merge
- ✓ Use schema enforcement, evolution, history, time travel and OPTIMIZE
- ✓ Describe Unity Catalog and managed vs external tables

START HERE IF

Phases 1-5 (you need a DataFrame worth saving).

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS
A jargon-free explanation of the idea.

DECODE THE CODE
Meaning of every function, option and symbol in the code above (first time it appears).

TIP
A habit that makes your code cleaner, faster or safer.

WATCH OUT
A trap that gives a wrong or surprising result.

NOTEBOOK OBSERVATION
Something specific noticed in the course notebook.

DATA ENGINEERING LENS
How the skill is used in real pipelines (summary page at the end).

EXTRA
Extra knowledge that is not in the notebook.

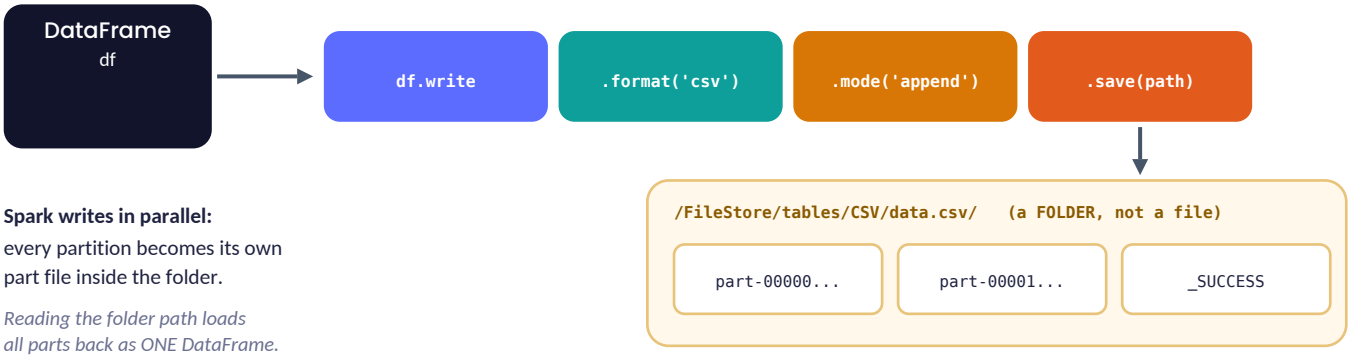
BEYOND THE NOTEBOOK
A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

6.1 The Spark Writer API . . . 3
6.2 Writing a CSV . . . 3
6.3 Data writing modes . . . 4
6.4 Parquet file format . . . 6
6.5 Managed vs external tables . . . 7
6.6 Delta Lake vs Parquet vs CSV . . . 8
6.7 ACID and the Delta transaction log . . . 9
6.8 Writing with save() vs saveAsTable() . . . 11
6.9 Append, overwrite and merge . . . 12
6.10 Schema enforcement vs schema evolution . . . 14
6.11 History, time travel and OPTIMIZE . . . 16
6.12 Databricks fundamentals: Unity Catalog, managed vs external, paths vs tables . . . 18
From notebook to pipeline . . . 22
Glossary . . . 23
Check yourself . . . 26
Answers and solutions . . . 27

6.1 The Spark Writer API

Writing mirrors reading. Where reading started with `spark.read`, writing starts with `df.write`: choose a **format**, a **mode** (what to do if the destination exists), then **save**.



Spark writes in parallel:
every partition becomes its own part file inside the folder.

Reading the folder path loads all parts back as ONE DataFrame.

PYSPARK · GENERAL SHAPE (SKELETON)

```
df.write.format('csv')          # 1. file format
        .mode('append')         # 2. what to do if data already exists
        .option('path', '...')  # 3. destination (or pass it to save())
        .save()                 # 4. write it - this is the action
```

DECODE THE CODE

`df.write`	Starts a **DataFrameWriter** - the mirror image of spark.read.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.mode('append')`	What to do if the destination already exists: error, append, overwrite or ignore.
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`.option('path', ...)`	The destination to write to (an alternative to passing it to save()).
`.save(path)`	Performs the write. This is the **action** that makes the job run.

A FOLDER, NOT A FILE

Spark writes in parallel: every partition becomes its own file. So `save('/FileStore/tables/CSV/data.csv')` creates a **folder named data.csv**, and inside it several `part-0000...` files plus small marker files such as `_SUCCESS`. Reading the folder path loads them all back as one DataFrame.

6.2 Writing a CSV

PYSPARK

```
df.write.format('csv')\
        .save('/FileStore/tables/CSV/data.csv')
```

DECODE THE CODE

`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.

No mode was given, so Spark uses the default, **error**: it works because the folder does not exist yet. Run the cell a second time and it fails, because by then the folder exists.

HEADER ROW

PYSPARK · EXTRA

```
df.write.format('csv').option('header', True).save('/FileStore/tables/CSV/data.csv')
```

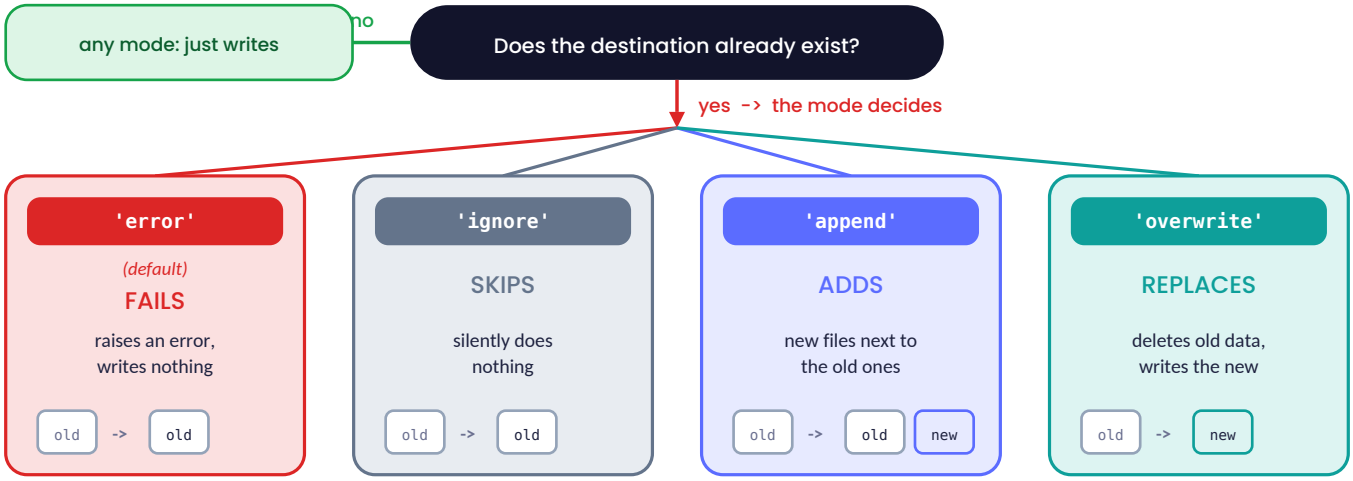
DECODE THE CODE

`header`	**True** = the first row of the CSV holds column names, not data.

The notebook's write does not set `header`, so the saved CSV files contain data rows only, without the column names. Add the option above if you want them.

6.3 Data writing modes

The **mode** decides what happens if the destination already exists. The notebook demonstrates all four, one cell each.



Run an append cell twice and the data is there twice - append is not idempotent. overwrite always leaves exactly one copy.
Folder state after the write (existing destination): old = data that was already there, new = the DataFrame you just wrote.

append

PYSPARK

```
df.write.format('csv')\
        .mode('append')\
        .save('/FileStore/tables/CSV/data.csv')
```

PYSPARK

```
df.write.format('csv')\
        .mode('append')\
        .option('path','/FileStore/tables/CSV/data.csv')\
        .save()
```

These two cells are equivalent: the destination can be passed straight to `save(path)` or set as `.option('path', path)` followed by an empty `save()`. Append **adds new files** next to the existing ones, so run it twice and the data is there twice.

overwrite

```pyspark
df.write.format('csv')\
.mode('overwrite')\
.option('path','/FileStore/tables/CSV/data.csv')\
.save()
```

Deletes whatever is at the destination and writes the new data - you end up with exactly one copy.

error

```pyspark
df.write.format('csv')\
.mode('error')\
.option('path','/FileStore/tables/CSV/data.csv')\
.save()
```

The default behaviour (also spelled `'errorifexists'`). If the destination exists, Spark **raises an error** and writes nothing. In the notebook this cell is meant to fail - that is the demonstration.

ignore

```pyspark
df.write.format('csv')\
.mode('ignore')\
.option('path','/FileStore/tables/CSV/data.csv')\
.save()
```

If the destination exists, Spark **silently does nothing**: no error and no new data.

Mode	Destination does not exist	Destination already exists
`'error'` / `'errorifexists'` (default)	Writes	**Fails** with an error
`'append'`	Writes	**Adds** the new data beside the old
`'overwrite'`	Writes	**Replaces** the old data
`'ignore'`	Writes	**Does nothing**, silently

What the notebook's write cells do to the folder, in order (assuming the folder is new):

#	Cell	Effect on /FileStore/tables/CSV/data.csv
1	plain `save(path)`	Folder created with one copy of the data (default mode, folder was new).
2	`append` with `save(path)`	Second copy added - data now present twice.
3	`append` with `.option('path')`	Third copy added.
4	`overwrite`	Everything deleted, one fresh copy written.
5	`error`	Folder exists, so the cell **fails** (expected).
6	`ignore`	Folder exists, so nothing happens.
7	`parquet` + `overwrite`	Old CSV files replaced by Parquet files (next section).

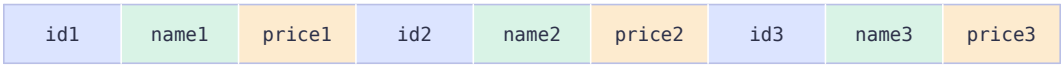
APPEND IS NOT IDEMPOTENT

Re-running an append cell duplicates data every time. If a job might be re-run, use overwrite, or design the pipeline so each run writes to its own place.

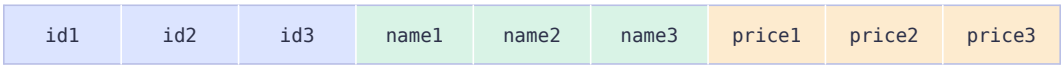
6.4 Parquet file format

Parquet is Spark's favourite file format. CSV stores data **row by row** as plain text; Parquet stores it **column by column** in a compressed binary file. Picture three rows of (id, name, price):

CSV (ROW-ORIENTED): VALUES OF ONE ROW SIT TOGETHER



PARQUET (COLUMN-ORIENTED): VALUES OF ONE COLUMN SIT TOGETHER



	CSV	Parquet
Layout	Text, row by row	Binary, column by column
Size	Large	Much smaller (compressed)
Schema	Not stored - must be inferred or supplied	Stored inside the file
Reading some columns	Has to read every column	Reads **only the columns you use**
Human readable	Yes	No

PYSPARK

```
df.write.format('parquet')\
.mode('overwrite')\
.option('path','/FileStore/tables/CSV/data.csv')\
.save()
```

NOTEBOOK OBSERVATION

The notebook writes Parquet into the same path it used for the CSV, **which is still called data.csv**. After the overwrite that folder contains Parquet files, so the name is misleading. In your own projects give Parquet outputs their own folder name.

READING PARQUET BACK

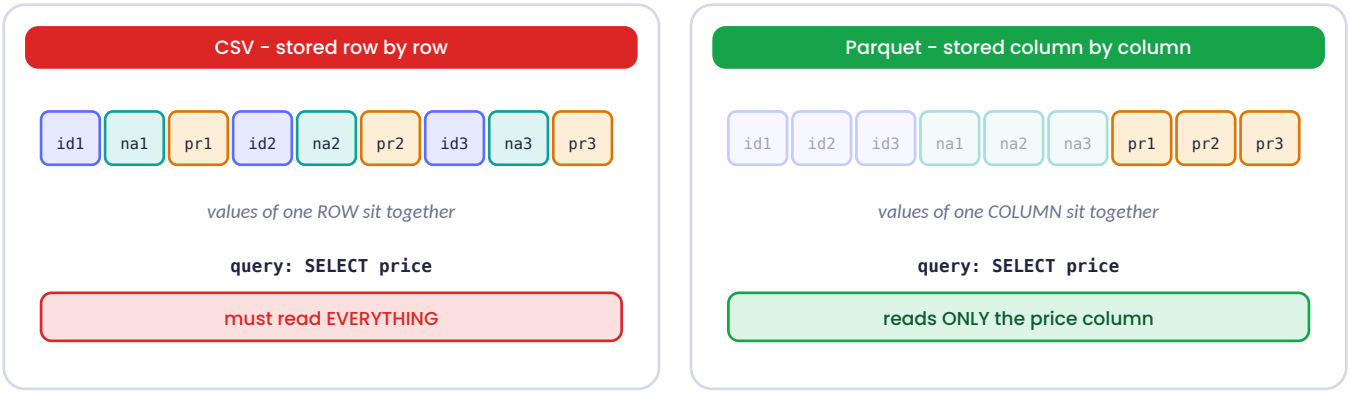
PYSPARK · EXTRA

```
df_p = spark.read.format('parquet').load('/FileStore/tables/CSV/data.csv')
```

DECODE THE CODE

`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)

No schema or header options are needed because Parquet carries its own schema.



6.5 Managed vs external tables

So far the data was saved as files at a path you chose. You can also save it as a **table** with a name, which you can then query by name from SQL.

PYSPARK

```
df.write.format('parquet')\
.mode('overwrite')\
.saveAsTable('my_table')
```

DECODE THE CODE

`.saveAsTable('name')`	Saves the data as a **named table** in Spark's catalog.

PYSPARK

```
df.display()
```

DECODE THE CODE

`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

No path is given, so Spark decides where the files go (the workspace's default warehouse location) and records the table name in its catalog. That makes `my_table` a **managed table**.

	Managed table	External table
Who owns the data files?	**Spark** - it picks the location and manages the files	**You** - the files stay at the path you provide
How to create (as in this notebook)	`.saveAsTable('my_table')` with **no path**	`.option('path', '...')` + `.saveAsTable(...)`
DROP TABLE removes	The table **and its data**	**Only the table's name** (metadata); the files remain
Best when	Spark owns the whole lifecycle of the data	Data is shared with other tools, or must survive the table

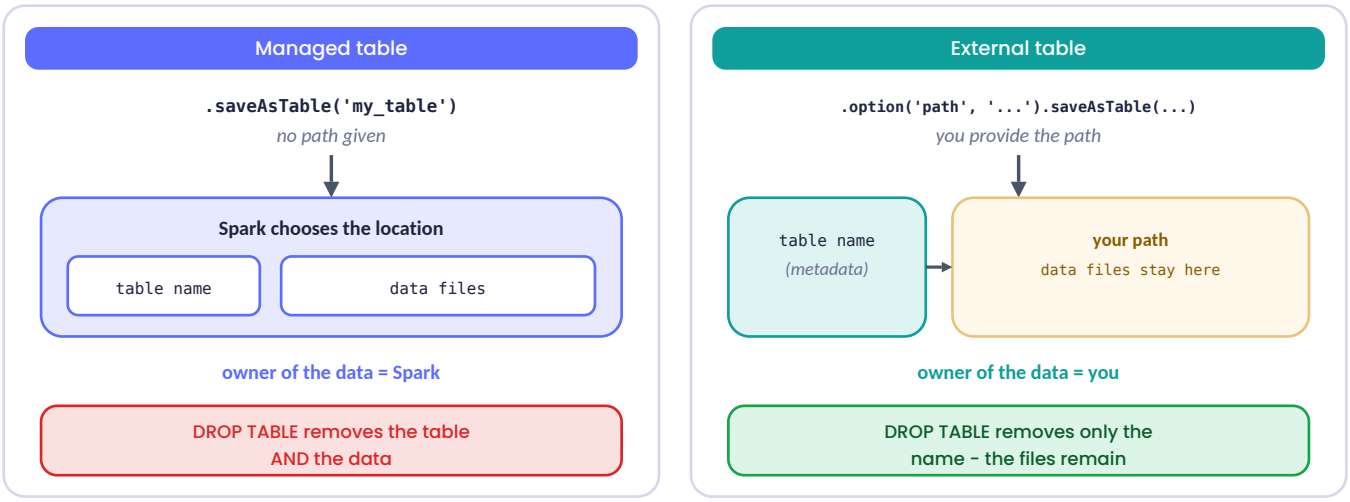
MAKING IT EXTERNAL

PYSPARK · EXTRA

```
df.write.format('parquet')\
  .mode('overwrite')\
  .option('path', '/FileStore/tables/external_table')\
  .saveAsTable('my_ext_table')
```

DROPPING DATA BY ACCIDENT

Think before running `DROP TABLE` on a managed table: the data goes with it. For data you cannot afford to lose, an external table is the safer shape.



6.6 Delta Lake vs Parquet vs CSV

BEYOND THE NOTEBOOK

Phase 6 wrote CSV and Parquet. Both are just **files**. **Delta Lake** is Parquet files **plus a transaction log**, and that log is what turns a folder of files into a reliable table.

	CSV text, row by row	Parquet binary, column by column	Delta Parquet files + a log
Compressed, columnar storage	no	yes	yes
Schema stored with the data	no	yes	yes
ACID transactions (all-or-nothing writes)	no	no	yes
Schema enforcement on write	no	no	yes
UPDATE / DELETE / MERGE	no	no	yes
Time travel (read an older version)	no	no	yes

Delta is not a new file format. Its data files ARE Parquet; the extra _delta_log folder is what adds the guarantees.
On Databricks, saveAsTable() creates a Delta table by default.

	CSV	Parquet	Delta
Layout	Text rows	Columnar binary	Parquet files + *_delta_log* folder
Schema stored?	No (header only)	Yes	Yes, and **enforced**
Types	Everything is text	Typed	Typed
Reads only needed columns?	No	Yes	Yes
Update / delete rows	No	No (rewrite the files)	**Yes** (UPDATE, DELETE, MERGE)
ACID transactions	No	No	**Yes**
Time travel / history	No	No	**Yes**
Safe with concurrent writers	No	No	Yes

ABOUT THE OUTPUTS IN THIS SECTION

Delta Lake examples and their outputs are **illustrative**: they follow the documented behaviour of Delta Lake / Databricks but were not executed while these notes were built. Run them in your own workspace to see the real values (version numbers, timestamps, file counts).

PYSPARK · WRITE A DELTA FOLDER

```
df.write.format('delta').mode('overwrite').save('/Volumes/main/demo/files/bigmart_delta')
```

DECODE THE CODE

`format('delta')`	Reads or writes in the **Delta Lake** format (Parquet files + transaction log).

WHAT THE FOLDER LOOKS LIKE (ILLUSTRATIVE)

```
bigmart_delta/
├── _delta_log/
│   └── 00000000000000000000.json       <- commit 0
├── part-00000-....snappy.parquet
└── part-00001-....snappy.parquet
```

DELTA IS A FORMAT, NOT A NEW ENGINE

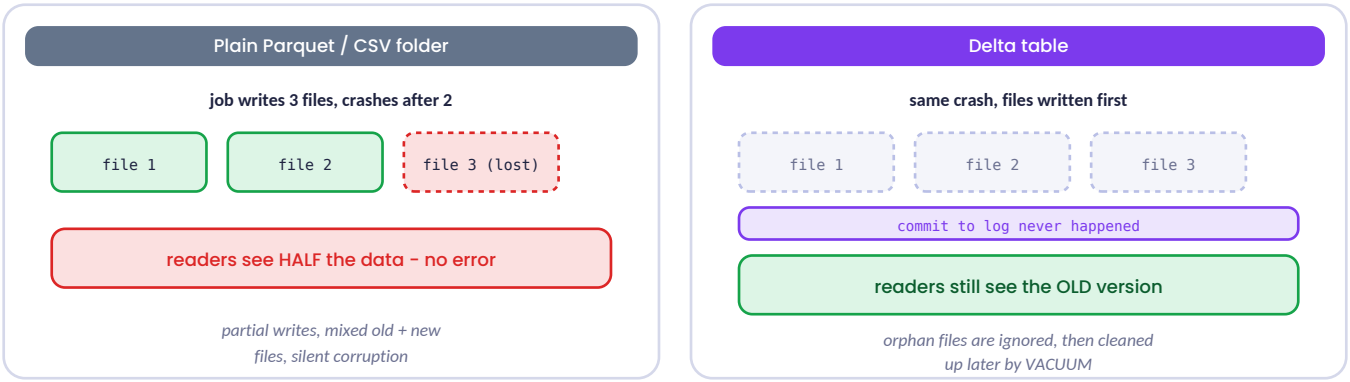
You still use Spark and the same `df.write` / `spark.read` calls. Changing `format('parquet')` to `format('delta')` is the whole code change. On Databricks, `saveAsTable` creates a Delta table by default.

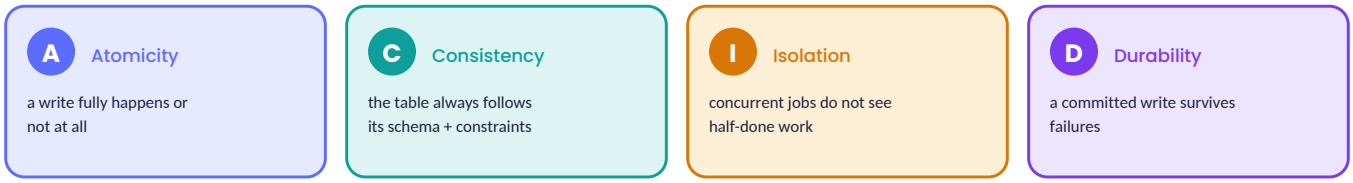
6.7 ACID and the Delta transaction log

BEYOND THE NOTEBOOK

ACID is the promise that a write is reliable. Plain files cannot give it: if a Spark job dies half-way through writing 50 files, readers see 20 of them. Delta gives all four properties:

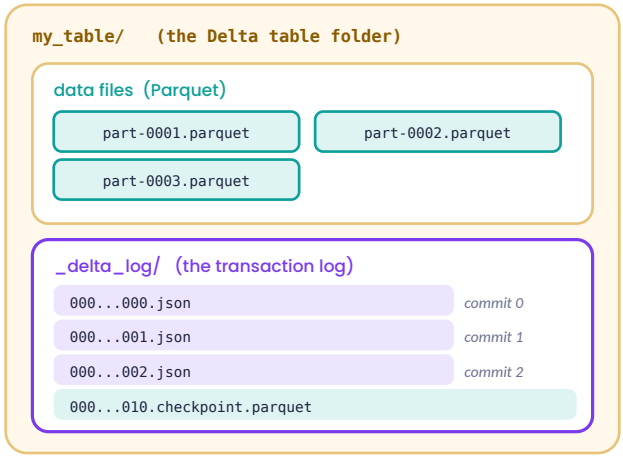
Letter	Meaning	What it prevents in Delta
Atomicity	A write fully happens or not at all.	A failed job leaving half-written output visible.
Consistency	Data always satisfies the table's rules (schema, constraints).	A wrong-typed column slipping in.
Isolation	Concurrent readers and writers do not see each other's partial work.	A dashboard reading while a load is running.
Durability	Once committed, it is stored safely.	Losing a committed write.





How it works: the _delta_log

Every change is recorded as a numbered **JSON commit** in the `_delta_log` folder: *add file x*, *remove file y*. A reader replays the log to learn which Parquet files make up the **current version**. A write becomes visible only when its commit file lands, and that single step is the atomic switch. Every 10 commits Delta also writes a **checkpoint** (a Parquet summary) so readers do not replay a long log.



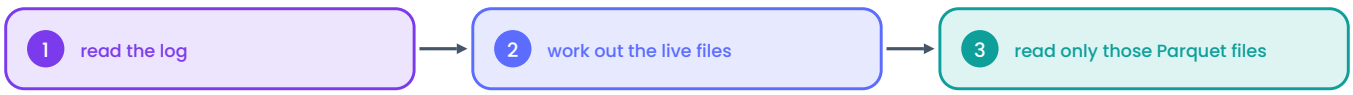
ONE COMMIT = ONE JSON FILE OF ACTIONS

```
{"commitInfo": {"operation": "WRITE"}}
{"add":    {"path": "part-0001.parquet"}}
{"add":    {"path": "part-0002.parquet"}}
{"remove": {"path": "part-0000.parquet"}}
# simplified: real entries also hold sizes, stats, schema
```

The TABLE = replay of the log

Files added minus files removed = what a reader sees.
A checkpoint file (every 10 commits) saves the replay so readers need not read every old JSON file.

WHAT A READER DOES



Illustrative layout. A file listed in a 'remove' action is ignored by readers even though it may still sit on disk.

THE LOG, CONCEPTUALLY (ILLUSTRATIVE)

```
00000000000000000000.json   add part-0, add part-1        (CREATE: version 0)
00000000000000000001.json   add part-2                     (APPEND: version 1)
00000000000000000002.json   remove part-1, add part-3      (UPDATE: version 2)
...
00000000000000000010.checkpoint.parquet                    (summary of versions 0-10)
```

WHY DATA ENGINEERS CARE

A failed run never leaves a half table; a rerun does not corrupt existing data; two jobs can write safely; and every version stays queryable (6.11). This is the foundation of the medallion pipeline in Phase 7.

TWO WRITERS CAN STILL CONFLICT

Delta uses optimistic concurrency: if two writers change the same files, one fails with a conflict error and must retry. Safe does not mean no errors.

6.8 Writing with save() vs saveAsTable()

BEYOND THE NOTEBOOK

Both write data. The difference is whether the result is just **files at a path** or a **named table** in the catalog.



WHY THE NAME USUALLY WINS IN PRODUCTION

- permissions are granted on the table
- lineage + discovery know the table
- renames / moves do not break readers

	save('path')	saveAsTable('name')
Writes	Files in the folder you give	Files **and** a catalog entry
How you read it back	`spark.read.format('delta').load('path')`	`spark.table('name')` or `SELECT ... FROM name`
SQL-friendly?	Not by name (only via `delta.`path``)	Yes
Permissions / lineage	Path-level (storage)	Table-level, governed by Unity Catalog
Managed?	No - you own the folder	Managed by default (no path); external if you give a path

PYSPARK · THE TWO WAYS (ILLUSTRATIVE)

```
# 1. files at a path
df.write.format('delta').mode('overwrite').save('/Volumes/main/demo/files/bigmart_delta')
spark.read.format('delta').load('/Volumes/main/demo/files/bigmart_delta')

# 2. a registered table (catalog.schema.table)
df.write.format('delta').mode('overwrite').saveAsTable('main.demo.bigmart')
spark.table('main.demo.bigmart')
```

DECODE THE CODE

`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.table('name')`	Reads a registered table as a DataFrame.

RULE OF THUMB

Use **tables** for data other people or jobs will query, and **paths** for raw files and exports. In Unity Catalog the table name has three parts: `catalog.schema.table`.

6.9 Append, overwrite and merge

BEYOND THE NOTEBOOK

Phase 6.3 covered `append`, `overwrite`, `error` and `ignore`. With Delta there is a fifth, much more useful operation: **MERGE** (also called **upsert** = update + insert).

TARGET (DELTA TABLE)

Item_Id	MRP
FDA15	249.8
DRC01	48.3
FDN15	141.6

SOURCE (NEW BATCH)

Item_Id	MRP
FDA15	259.9
DRC01	48.3
FDX07	182.4

```
MERGE ... ON t.Item_Id = s.Item_Id

WHEN MATCHED
   -> UPDATE
WHEN NOT MATCHED
   -> INSERT
WHEN NOT MATCHED BY SOURCE
   -> DELETE (optional)
```

TARGET AFTER THE MERGE

Item_Id	MRP	action
FDA15	259.9	updated
DRC01	48.3	same
FDN15	141.6	untouched
FDX07	182.4	inserted

FDN15 is not in the source, so it is kept (no 'not matched by source' rule).

THREE WAYS TO PUT A NEW BATCH INTO A DELTA TABLE

append
adds all source rows
FDA15 appears TWICE

overwrite
replaces the whole table
FDN15 is LOST

MERGE
updates matches, inserts new
one row per key

MERGE is the standard 'upsert'. Run it twice with the same batch and the table ends the same - it is safe to re-run.

Operation	What it does	Re-run safe?	Use for
append	Adds the new rows.	**No** - duplicates	Immutable events / logs, with a guard.
overwrite	Replaces the whole table (or only matching partitions with `replaceWhere`).	Yes	Full reloads, small tables.
merge	Updates rows that match on a key, inserts the ones that do not.	**Yes**	Incremental loads of changing data.

PYSPARK · MERGE (UPSERT) WITH THE PYTHON API (ILLUSTRATIVE)

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, 'main.demo.bigmart')

(target.alias('t')
       .merge(updates.alias('s'),
              't.Item_Identifier = s.Item_Identifier '
              'AND t.Outlet_Identifier = s.Outlet_Identifier')
       .whenMatchedUpdateAll()          # key exists  -> update the row
       .whenNotMatchedInsertAll()       # key is new   -> insert the row
       .execute())
```

DECODE THE CODE

`DeltaTable.forName(...)`	Python handle on a Delta table, used for merge, update, delete and history.
`.alias('new')`	Gives a column a new name **in this result only**.
`.merge(source, condition)`	Starts an upsert: match source rows to the table on a condition.
`.whenMatchedUpdateAll()`	If the key exists, update the row with all the source columns.
`.whenNotMatchedInsertAll()`	If the key is new, insert the source row.
`.execute()`	Runs the merge - the action.

SQL · THE SAME MERGE (ILLUSTRATIVE)

```
MERGE INTO main.demo.bigmart AS t
USING updates AS s
  ON t.Item_Identifier = s.Item_Identifier AND t.Outlet_Identifier = s.Outlet_Identifier
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

DECODE THE CODE

`MERGE INTO`	Upsert: update matching rows, insert the rest.
`WHEN [NOT] MATCHED`	What to do for rows that do / do not match on the key.

THE SOURCE MUST HAVE ONE ROW PER KEY

If two source rows match the same target row, MERGE fails (a multiple-match error). Deduplicate the source first, for example keep the latest row per key with `row_number()` from Phase 5.

PYSPARK · DEDUPLICATE THE SOURCE BEFORE MERGE

```
w = Window.partitionBy('Item_Identifier', 'Outlet_Identifier').orderBy(col('updated_at').desc())
updates = updates.withColumn('rn', row_number().over(w)).filter(col('rn') == 1).drop('rn')
```

DECODE THE CODE

`Window.partitionBy(...)`	A window per group (Phase 5) - here used to keep the latest row per key.
`Window`	Defines the group of rows a window function looks at.
`Window.partitionBy(col)`	Splits the data into separate groups first; the window restarts in each group.
`.orderBy(col)`	Sort order **inside** the window - needed for numbering and running totals.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`.desc() / .asc()`	Sort direction for one column: descending / ascending.
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`row_number()`	Numbers rows 1, 2, 3 ... in window order. Always unique.
`.over(window)`	Applies the function across the window instead of collapsing rows.
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.

LINK TO THE NOTEBOOK

The notebook shows **append** duplicating data when a cell is run twice. MERGE is the standard cure: running it twice with the same source leaves the table unchanged (idempotent, Phase 7.9).

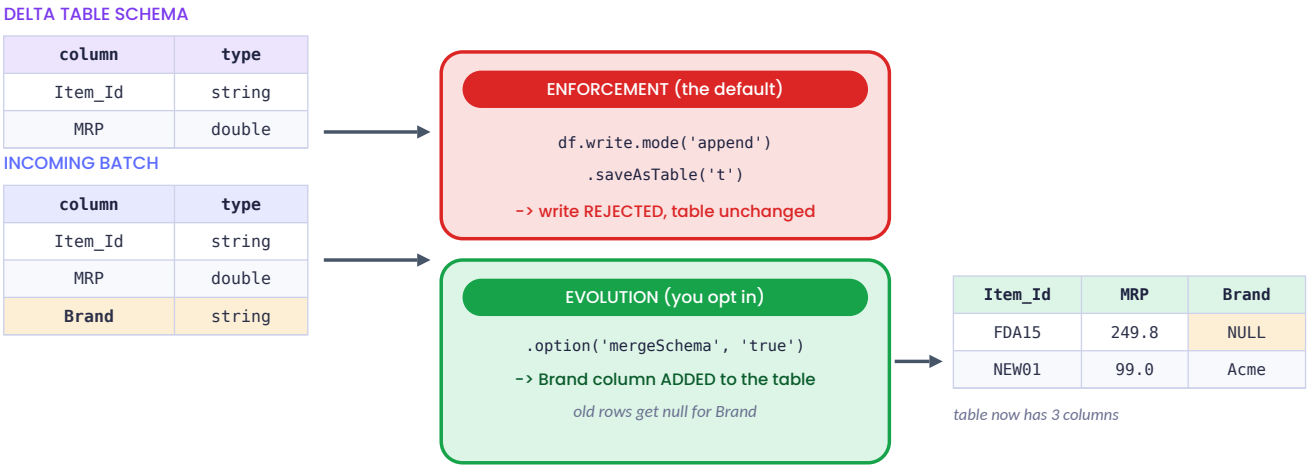
REPLACEWHERE

To overwrite only a slice, for example one day: `.option('replaceWhere', "load_date = '2026-10-15'")` with `mode('overwrite')`. Rows outside the condition are untouched.

6.10 Schema enforcement vs schema evolution

BEYOND THE NOTEBOOK

Delta **enforces** the table's schema on every write: a DataFrame with an unexpected column or an incompatible type is rejected instead of silently corrupting the table. **Evolution** is the deliberate, opt-in way to let the schema change.



Evolution is not a free pass: it makes drift silent. Pair it with a validation step that decides WHICH new columns you accept.
A rename or type change needs overwriteSchema (a rewrite) - never mergeSchema.

	Schema enforcement	Schema evolution
Default?	**Yes**	No - you opt in
New column in the DataFrame	Write **fails** (AnalysisException: schema mismatch).	Column is **added** to the table; old rows get null.
How	Nothing to do.	`.option('mergeSchema', True)` on append, or MERGE ... WITH SCHEMA EVOLUTION.
Replace the whole schema	-	`.option('overwriteSchema', True)` with mode overwrite
Best for	Protecting Silver / Gold tables.	Landing raw (Bronze) data that gains columns.

PYSPARK · ENFORCEMENT, THEN EVOLUTION (ILLUSTRATIVE)

```
new = df.withColumn('Promo_Flag', lit(1))                # one extra column

new.write.format('delta').mode('append').saveAsTable('main.demo.bigmart')   # FAILS: mismatch

new.write.format('delta').mode('append')\
   .option('mergeSchema', True).saveAsTable('main.demo.bigmart')            # OK: column added
```

DECODE THE CODE

`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).
`mergeSchema`	Write option: let new columns be added to the Delta table (schema evolution).

EVOLVE ON PURPOSE

Automatic evolution is convenient but lets upstream mistakes flow downstream. Pair it with the schema check from section 1.8: detect the drift, decide, then evolve (or reject).

SQL · DELTA CONSTRAINTS ARE DATA-QUALITY RULES THE TABLE ENFORCES (ILLUSTRATIVE)

```
ALTER TABLE main.demo.bigmart ADD CONSTRAINT mrp_positive CHECK (Item_MRP > 0);
ALTER TABLE main.demo.bigmart ALTER COLUMN Item_Identifier SET NOT NULL;
```

DECODE THE CODE

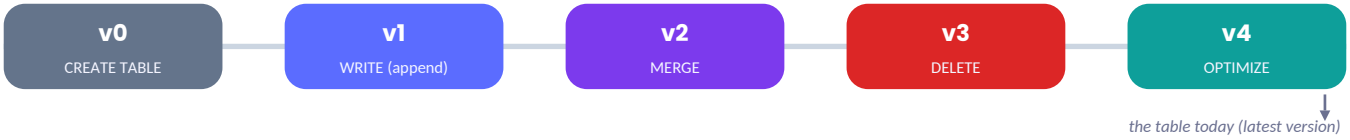
`ADD CONSTRAINT / SET NOT NULL`	Rules the Delta table enforces on every write.

A write that violates a **CHECK** or **NOT NULL** constraint fails as a whole (atomically). This is the table-level version of the checks from section 4.12: **fail fast**, built in.

6.11 History, time travel and OPTIMIZE

BEYOND THE NOTEBOOK

Because the log keeps every commit, Delta can show **what changed** and read the table **as it was**.



DESCRIBE HISTORY MY_TABLE (NEWEST FIRST)

version	operation	operationMetrics (excerpt)
4	OPTIMIZE	numFilesAdded: 1 numFilesRemoved: 7
3	DELETE	numDeletedRows: 12
2	MERGE	numTargetRowsUpdated: 3 numTargetRowsInserted: 2
1	WRITE	mode: Append numOutputRows: 40
0	CREATE TABLE	-

READ OR UNDO AN OLDER VERSION

```
SELECT * FROM t VERSION AS OF 2
```
read-only look at v2

```
spark.read.option('versionAsOf',
2).table('t')
```
same, from Python

```
RESTORE TABLE t TO VERSION AS OF 2
```
make v2 the new latest (creates v5)

Time travel works only while the old data files still exist: VACUUM (default keeps 7 days) deletes them for good.

SQL · THE COMMIT HISTORY

```sql
DESCRIBE HISTORY main.demo.bigmart
```

DECODE THE CODE

`DESCRIBE HISTORY`	Lists every commit of a Delta table: version, time, operation.

OUTPUT · ILLUSTRATIVE (real history has more columns, such as userName and metrics)

version	timestamp	operation	operationParameters
2	2026-10-16 09:14	MERGE	predicate: t.Item_Identifier = ...
1	2026-10-16 09:10	WRITE	mode: Append
0	2026-10-16 09:05	CREATE TABLE AS SELECT	-

SQL · READ AN OLD VERSION, OR PUT THE TABLE BACK (ILLUSTRATIVE)

```sql
SELECT * FROM main.demo.bigmart VERSION AS OF 1
SELECT * FROM main.demo.bigmart TIMESTAMP AS OF '2026-10-16 09:10:00'
RESTORE TABLE main.demo.bigmart TO VERSION AS OF 1
```

DECODE THE CODE

`select ... from`	Pick columns **from** a table or view.
`VERSION AS OF`	Time travel: query the table as it was.
`RESTORE TABLE`	Roll a Delta table back to an earlier version.

PYSPARK · THE SAME IN PYTHON (ILLUSTRATIVE)

```python
spark.read.format('delta').option('versionAsOf', 1).table('main.demo.bigmart')
```

DECODE THE CODE

`versionAsOf`	Read option: load the table as of a given version (time travel).

WHAT TIME TRAVEL IS GOOD FOR

Debugging: what did the table look like before last night's load?

Audit: who changed what, and when (the history).

Recovery: `RESTORE` after a bad overwrite or delete.

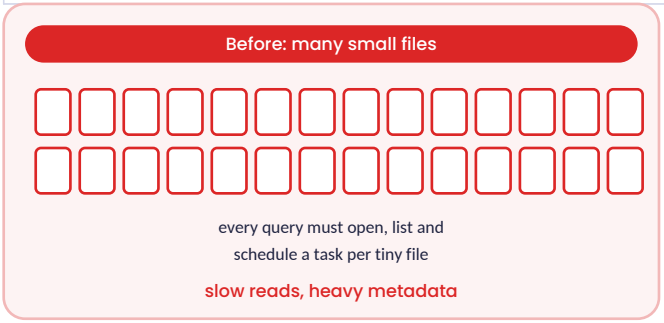
Reproducibility: re-run a report on the exact data it first used.

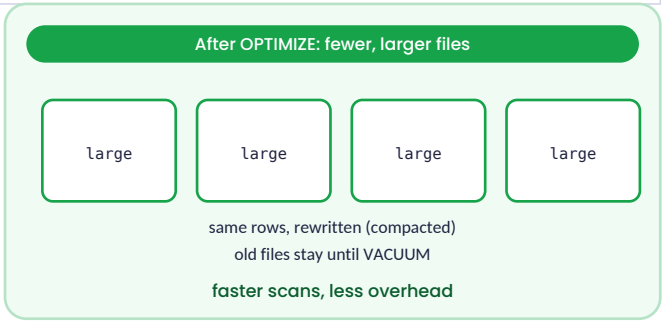
HISTORY IS NOT A BACKUP FOREVER

Old data files are removed by `VACUUM` (default retention 7 days) and the log is kept for 30 days by default, so time travel only reaches back that far unless you change the table settings.

OPTIMIZE and VACUUM

Command	What it does	Why
`OPTIMIZE table`	Compacts many small files into fewer, larger ones.	Frequent appends create many small files; reads slow down.
`OPTIMIZE table ZORDER BY (col)`	Also co-locates rows with similar values of a column.	Faster filters on that column (data skipping).
`VACUUM table`	Deletes data files no longer referenced and older than the retention period.	Saves storage. Time travel before that point is lost.






```
OPTIMIZE my_table                -- compact small files (a new version in the history)
VACUUM my_table DRY RUN          -- list files that would be deleted; deletes nothing
VACUUM my_table                  -- delete unreferenced files older than the retention (7 days)
```

GOOD TO KNOW

Databricks predictive optimization can run OPTIMIZE and VACUUM for Unity Catalog managed tables.

OPTIMIZE ... ZORDER BY or liquid clustering also group related rows - a next step.

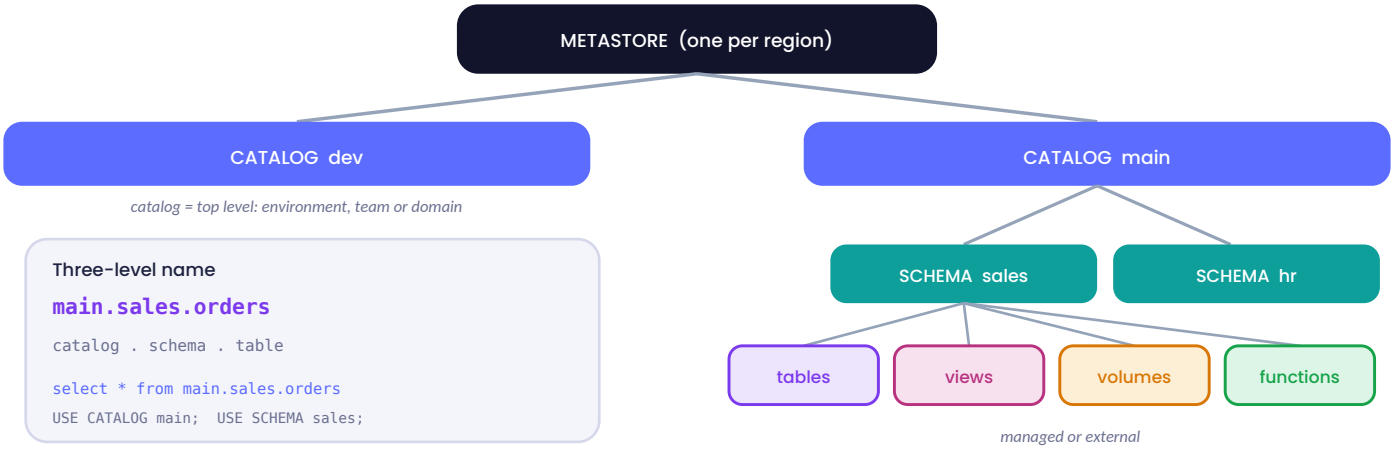
BASICS ONLY

On Databricks, newer features such as liquid clustering and predictive optimization can replace manual ZORDER and scheduled OPTIMIZE. For this course it is enough to know *why* small files hurt and that OPTIMIZE compacts them.

6.12 Databricks fundamentals: Unity Catalog, managed vs external, paths vs tables

BEYOND THE NOTEBOOK

Unity Catalog is Databricks' central governance layer: one place to define data, who can use it, and how it is related. Every table or volume has a **three-level name** inside a metastore.



Three-level name

`main.sales.orders`

`catalog . schema . table`

```
select * from main.sales.orders
USE CATALOG main;  USE SCHEMA sales;
```

Files live in VOLUMES:

`/Volumes/<catalog>/<schema>/<volume>/<path>`

`e.g. /Volumes/main/sales/raw/BigMart_Sales.csv (replaces the old /FileStore/tables/ habit)`

Level	What it holds	Example
Metastore	The top container for a region / account.	(usually one per region)
Catalog	A collection of schemas, often per environment or team.	`main`, `dev`, `prod`
Schema (database)	A collection of tables, views, volumes and functions.	`main.sales`
Table / view	Structured data (Delta by default).	`main.sales.bigmart`
Volume	A governed folder for **files**: CSV, JSON, images, exports.	`/Volumes/main/sales/raw/`

PYSPARK · NAMES AND PATHS IN UNITY CATALOG (ILLUSTRATIVE)

```
spark.sql('USE CATALOG main')
spark.sql('USE SCHEMA sales')
spark.table('main.sales.bigmart')                    # full three-part name
spark.read.csv('/Volumes/main/sales/raw/BigMart_Sales.csv', header=True)
```

DECODE THE CODE

`spark.sql("...")`	Runs a SQL query from Python and returns a **DataFrame**.
`USE CATALOG / SCHEMA`	Sets the default catalog / schema so you can use short table names.

SQL · PERMISSIONS ARE GRANTED ON CATALOG OBJECTS (ILLUSTRATIVE)

```
GRANT SELECT ON TABLE main.sales.bigmart TO `analysts`;
```

DECODE THE CODE

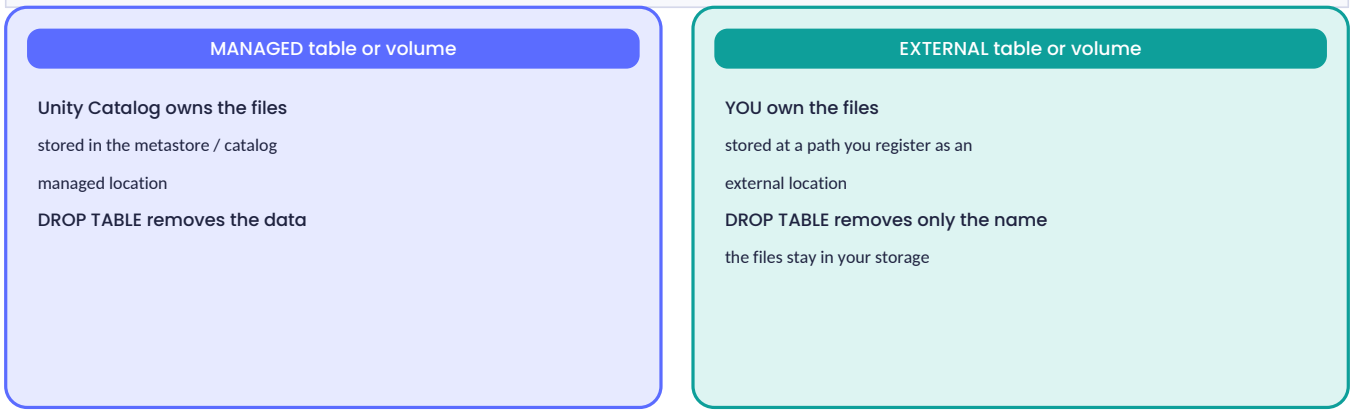
`GRANT`	Gives a user or group a permission on a table or schema.

NOTEBOOK VS UNITY CATALOG

The notebook reads from `/FileStore/tables/...` and calls `saveAsTable('my_table')` - the older, workspace-level style. In a Unity Catalog workspace you would keep raw files in a **Volume** and name tables `catalog.schema.table`. Free Edition restricts the public DBFS root, so Volumes are the supported place for files.

Managed vs external in Unity Catalog

	Managed table	External table
Where the files live	In storage Unity Catalog manages for the schema / catalog.	In a storage location **you** own and register (an external location).
DROP TABLE	Removes the table **and** its data files (after a retention period).	Removes only the table definition; **files stay**.
Created by	`saveAsTable('c.s.t')` with no path.	`CREATE TABLE ... LOCATION '...'` or `.option('path', ...)`.
Choose when	Databricks should own the lifecycle (the usual choice).	Other tools must read/write the same files, or the data must outlive the table.



Managed is the default and the simplest. Choose external when other systems must keep using the same files.

Delta storage path vs registered table

	Path: delta.`/some/path`	Registered table: catalog.schema.table
Is it in the catalog?	No - just a folder of Delta files.	Yes.
Permissions	Whoever can access the storage path.	Table-level grants, lineage, auditing.
Rename / move	You manage the path in every notebook.	Code uses the name; the location can change.
Typical use	Quick checks, external tools, bronze file drops.	Everything the team shares.

	Delta storage PATH	Registered TABLE
Created with	`df.write ... .save('/path')`	`df.write ... .saveAsTable('c.s.t')`
How you read it	`load('/path')` or `delta.`/path``	`spark.table('c.s.t')` or `SQL name`
Registered in the catalog?	No	Yes
Permissions	On the storage location / volume	On the table (GRANT SELECT ...)
Lineage + discovery	Weak: files, not objects	Tracked per table and column
Move / rename the folder	Breaks every reader	Readers keep using the name
Best for	Raw landing files, quick tests	Anything shared or scheduled

Both are Delta tables on disk. The difference is whether the catalog knows about them.

SQL · PATH vs NAME (ILLUSTRATIVE)

```sql
SELECT * FROM delta.`/Volumes/main/sales/delta/bigmart`      -- query a path directly
SELECT * FROM main.sales.bigmart                             -- query the registered table
```

DECODE THE CODE

`delta.`path``	Query a Delta folder directly by path.

FREE EDITION NOTE

Free Edition is serverless-only and includes Unity Catalog. You can practise every example in this phase there, except cache/persist (4.8) and anything that needs the old DBFS root.

INTERVIEW CORNER

Q What is Delta Lake?

A An open table format that stores data as Parquet files plus a transaction log, adding ACID transactions, schema enforcement, time travel and efficient updates and merges to a data lake.

Q How does Delta provide atomic writes?

A Data files are written first; the write only becomes visible when a single JSON commit file is added to the _delta_log. Readers replay the log, so a failed job leaves no partial result.

Q Delta vs Parquet?

A Parquet is only a file format: no transactions, no row-level update/delete, no history. Delta is Parquet plus a log, so it adds reliability, MERGE, schema enforcement and time travel.

Q save() vs saveAsTable()?

A save() writes files at a path. saveAsTable() also registers the table in the catalog so it can be queried by name, governed and shared.

Q Append, overwrite, merge - which do you choose for a daily incremental load?

A Merge (upsert) on the business key, because it is idempotent: re-running it does not duplicate rows. Append is only safe if you guard against re-runs.

Q What happens if you write a DataFrame with an extra column to a Delta table?

A It fails (schema enforcement) unless you opt in with mergeSchema or MERGE schema evolution.

INTERVIEW CORNER

Q How do you undo a bad overwrite?

A DESCRIBE HISTORY to find the good version, then RESTORE TABLE ... TO VERSION AS OF n, provided the files have not been vacuumed.

Q What does OPTIMIZE do, and VACUUM?

A OPTIMIZE compacts small files into larger ones (optionally ZORDER). VACUUM deletes unreferenced files older than the retention period, which limits time travel.

Q Managed vs external table?

A Managed: the platform owns the files and DROP removes data. External: you own the location and DROP removes only the metadata.

Q Unity Catalog hierarchy?

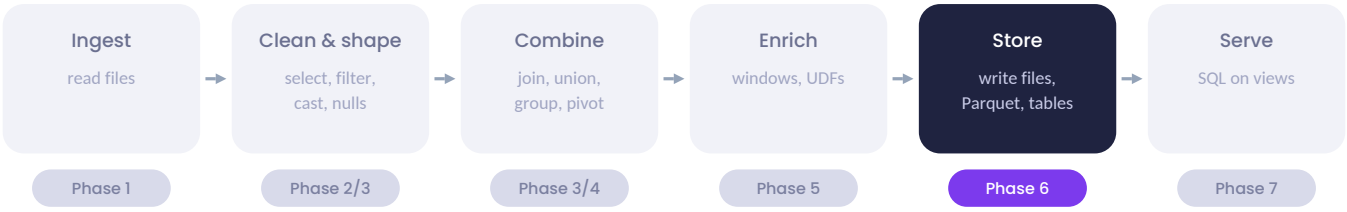
A Metastore > Catalog > Schema > Table / View / Volume / Function, with permissions and lineage defined centrally.

PHASE 6 CHECKPOINT

- I can write a DataFrame with `df.write.format(...).mode(...).save(...)` and I know it creates a folder of part files.
- I can name all four modes and say exactly what each does when the destination exists.
- I know append duplicates data when a cell is re-run.
- I can explain why Parquet is smaller and faster than CSV.
- I can tell a managed table from an external one and say what DROP TABLE does to each.
- I can explain what Delta adds to Parquet, and how the transaction log gives ACID guarantees.
- I can choose between `save` and `saveAsTable`, and between append, overwrite and merge.
- I know schema enforcement is the default and evolution is opt-in.
- I can read `DESCRIBE HISTORY`, time-travel to a version, and explain OPTIMIZE and VACUUM.
- I can name the Unity Catalog levels and tell managed from external, and paths from registered tables.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

Pick the **write mode** to match the job: overwrite for re-runnable (idempotent) jobs, append for incremental loads, error as a safety net, ignore to skip silently.

Append duplicates data if a job is re-run - plan for that in any pipeline that can be retried.

Parquet is the usual storage format: compressed, columnar, and it carries its own schema.

Managed vs external decides who owns the data: with an external table, dropping the table does not delete the files.

The output is a folder of part files; downstream readers read the folder path.

Beyond the notebook: **Delta** adds atomic writes, schema enforcement, MERGE and time travel - the foundation for re-runnable pipelines.

Prefer **registered tables** (catalog.schema.table) to loose paths so permissions and lineage work.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Write mode	What to do when the destination already exists: error, append, overwrite, ignore.
Part file	One output file per partition inside the output folder.
Idempotent	Running the job twice gives the same result as running it once.
Parquet	Compressed, columnar file format that stores its own schema.
Managed table	Table whose data Spark owns; dropping the table deletes the data.
External table	Table pointing at files you own; dropping it keeps the files.
Delta Lake	Parquet files plus a transaction log: ACID, MERGE, schema enforcement, time travel.
ACID	Atomicity, Consistency, Isolation, Durability.
_delta_log	Folder of numbered JSON commits that defines each table version.
MERGE (upsert)	Update matching rows and insert new ones in one atomic operation.
Schema enforcement / evolution	Reject mismatched writes by default / allow the schema to change on request.
Time travel	Reading a table as of an earlier version or timestamp.
OPTIMIZE / VACUUM	Compact small files / delete old unreferenced files.
Unity Catalog	Databricks' governance layer: metastore, catalog, schema, table, volume.
Volume	A governed folder for files inside Unity Catalog.

Code terms decoded in this phase

Code term	Meaning
`df.write`	Starts a **DataFrameWriter** - the mirror image of spark.read.
`.format('csv')`	Chooses the file format: **csv**, **json**, **parquet** ... (the same call is used when writing).
`.mode('append')`	What to do if the destination already exists: error, append, overwrite or ignore.
`.option(key, value)`	Sets **one** setting for the reader or writer. Call it again for each extra setting.
`.option('path', ...)`	The destination to write to (an alternative to passing it to save()).
`.save(path)`	Performs the write. This is the **action** that makes the job run.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`/FileStore/tables/`	The Databricks folder (DBFS) where uploaded files live.
`header`	**True** = the first row of the CSV holds column names, not data.
`spark.read`	Starts a **DataFrameReader**: the object you configure and then use to load a file.
`.load(path)`	Points Spark at the file or folder and returns a DataFrame. (Lazy until an action needs the data.)
`.saveAsTable('name')`	Saves the data as a **named table** in Spark's catalog.
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`format('delta')`	Reads or writes in the **Delta Lake** format (Parquet files + transaction log).
`spark`	The SparkSession - your entry point to Spark. Databricks creates it for you, already named **spark**.
`spark.table('name')`	Reads a registered table as a DataFrame.
`DeltaTable.forName(...)`	Python handle on a Delta table, used for merge, update, delete and history.
`.alias('new')`	Gives a column a new name **in this result only**.
`.merge(source, condition)`	Starts an upsert: match source rows to the table on a condition.
`.whenMatchedUpdateAll()`	If the key exists, update the row with all the source columns.
`.whenNotMatchedInsertAll()`	If the key is new, insert the source row.
`.execute()`	Runs the merge - the action.
`MERGE INTO`	Upsert: update matching rows, insert the rest.
`WHEN [NOT] MATCHED`	What to do for rows that do / do not match on the key.
`Window.partitionBy(...)`	A window per group (Phase 5) - here used to keep the latest row per key.
`Window`	Defines the group of rows a window function looks at.
`Window.partitionBy(col)`	Splits the data into separate groups first; the window restarts in each group.
`.orderBy(col)`	Sort order **inside** the window - needed for numbering and running totals.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).

Code term	Meaning
`.desc() / .asc()`	Sort direction for one column: descending / ascending.
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`row_number()`	Numbers rows 1, 2, 3 … in window order. Always unique.
`.over(window)`	Applies the function across the window instead of collapsing rows.
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).
`mergeSchema`	Write option: let new columns be added to the Delta table (schema evolution).
`ADD CONSTRAINT / SET NOT NULL`	Rules the Delta table enforces on every write.
`DESCRIBE HISTORY`	Lists every commit of a Delta table: version, time, operation.
`select ... from`	Pick columns **from** a table or view.
`VERSION AS OF`	Time travel: query the table as it was.
`RESTORE TABLE`	Roll a Delta table back to an earlier version.
`versionAsOf`	Read option: load the table as of a given version (time travel).
`spark.sql("...")`	Runs a SQL query from Python and returns a **DataFrame**.
`USE CATALOG / SCHEMA`	Sets the default catalog / schema so you can use short table names.
`GRANT`	Gives a user or group a permission on a table or schema.
`delta.`path``	Query a Delta folder directly by path.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 Which write mode fails when the destination already exists? Show answer

Q2 What does `mode('ignore')` do when the destination exists? Show answer

Q3 Why does `save('/FileStore/tables/CSV/data.csv')` create a folder? Show answer

Q4 Give two reasons Parquet is preferred over CSV. Show answer

Q5 What does `DROP TABLE` do to a managed table and to an external table? Show answer

Q6 What does Delta add to Parquet? Show answer

Q7 Why is MERGE preferred over append for a daily load? Show answer

Q8 A write with an extra column fails on a Delta table. Why, and how do you allow it? Show answer

Q9 How do you undo a bad overwrite? Show answer

Q10 Name the Unity Catalog levels. Show answer

Hands-on tasks

Task 1 Save df as Parquet to its own folder, replacing anything already there. Show solution

Task 2 Save df as a managed table called bigmart_tbl, then save it again as an external table called bigmart_ext. Show solution

Task 3 Upsert a DataFrame of updates into a Delta table on the key Item_Identifier. Show solution

Task 4 Show a table's history, then read version 0. Show solution

Answers and solutions

A1 `'error'` (also `'errorifexists'`), which is the default. back to question

A2 Nothing - silently. No error and no new data. back to question

A3 Spark writes in parallel: each partition becomes its own part file inside a folder named data.csv. back to question

A4 It is compressed (smaller) and column-oriented, so queries read only the columns they need; it also stores the schema inside the file. back to question

A5 Managed: removes the table **and** its data. External: removes only the table's metadata; the files remain. back to question

A6 A transaction log: ACID writes, schema enforcement, MERGE / UPDATE / DELETE, history and time travel. back to question

A7 It is idempotent: re-running it matches the same keys and changes nothing, whereas append duplicates rows. back to question

A8 Schema enforcement rejects it. Opt in with `option('mergeSchema', True)` (evolution). back to question

A9 `DESCRIBE HISTORY`, then `RESTORE TABLE ... TO VERSION AS OF n` - if the files were not vacuumed. back to question

A10 Metastore, catalog, schema, then table / view / volume. back to question

Task solutions

Solution 1 back to task

SOLUTION

```
df.write.format('parquet')\
    .mode('overwrite')\
    .option('path','/FileStore/tables/bigmart_parquet')\
    .save()
```

Solution 2 back to task

SOLUTION

```
df.write.format('parquet').mode('overwrite').saveAsTable('bigmart_tbl')

df.write.format('parquet').mode('overwrite')\
    .option('path','/FileStore/tables/bigmart_ext')\
    .saveAsTable('bigmart_ext')
```

Solution 3 back to task

SOLUTION

```
from delta.tables import DeltaTable

DeltaTable.forName(spark, 'main.demo.bigmart').alias('t')\
    .merge(updates.alias('s'), 't.Item_Identifier = s.Item_Identifier')\
    .whenMatchedUpdateAll()\
    .whenNotMatchedInsertAll()\
    .execute()
```

Solution 4 back to task

SOLUTION

```
DESCRIBE HISTORY main.demo.bigmart;

SELECT * FROM main.demo.bigmart VERSION AS OF 0;
```

PREVIOUS
← Phase 5 · Window Functions & UDFs

NEXT
Phase 7 · Spark SQL & Pipelines →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.